

Destructuring, Functional Programming & OOP

ADVANCED SOFTWARE ARCHITECTURE PRINCIPLES

01. DESTRUCTURING

Simplifying complex data access through concise syntax.

Pattern Matching Fundamentals



Object Unpacking

Extract properties directly into variables by name, significantly reducing repetitive dot notation.



Array Sequencing

Assign elements to variables based on their position, useful for handling tuples and spreadsheet data.



Nested Access

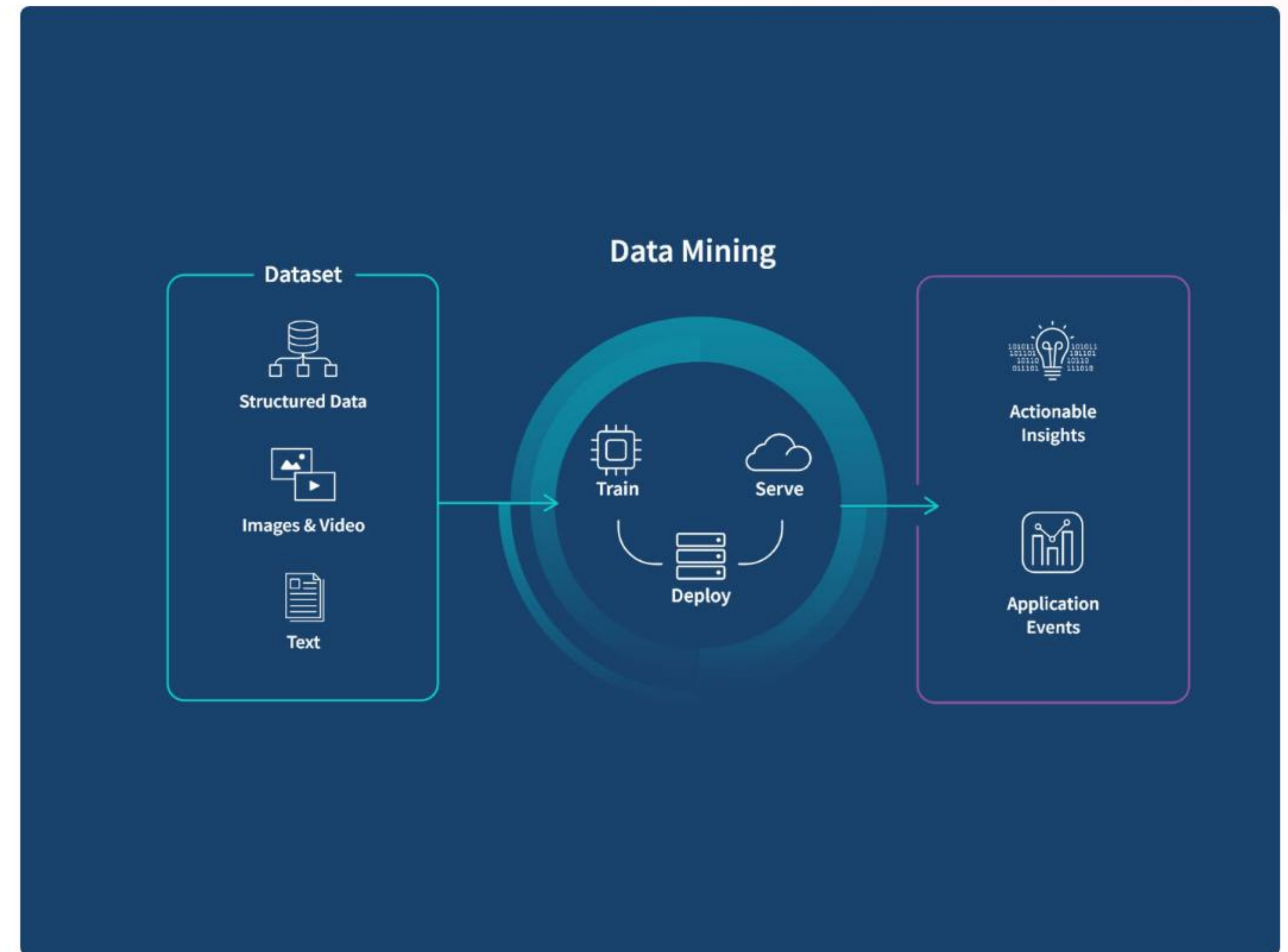
Reach deep into data structures in a single operation while maintaining code readability.

Practical Implementation

Destructuring allows us to "unpack" values from arrays or properties from objects into distinct variables.

```
// Object Destructuring
const { name, age } = user;

// Array Destructuring
const [first, second] = list;
```



02. FUNCTIONAL PROGRAMMING

Declarative logic for predictable and scalable codebases.

FP Core Principles



Pure Functions

Functions that produce identical outputs for identical inputs with zero side effects.



Immutability

Data state is never modified directly. We derive new state from existing data safely.



High-Order Logic

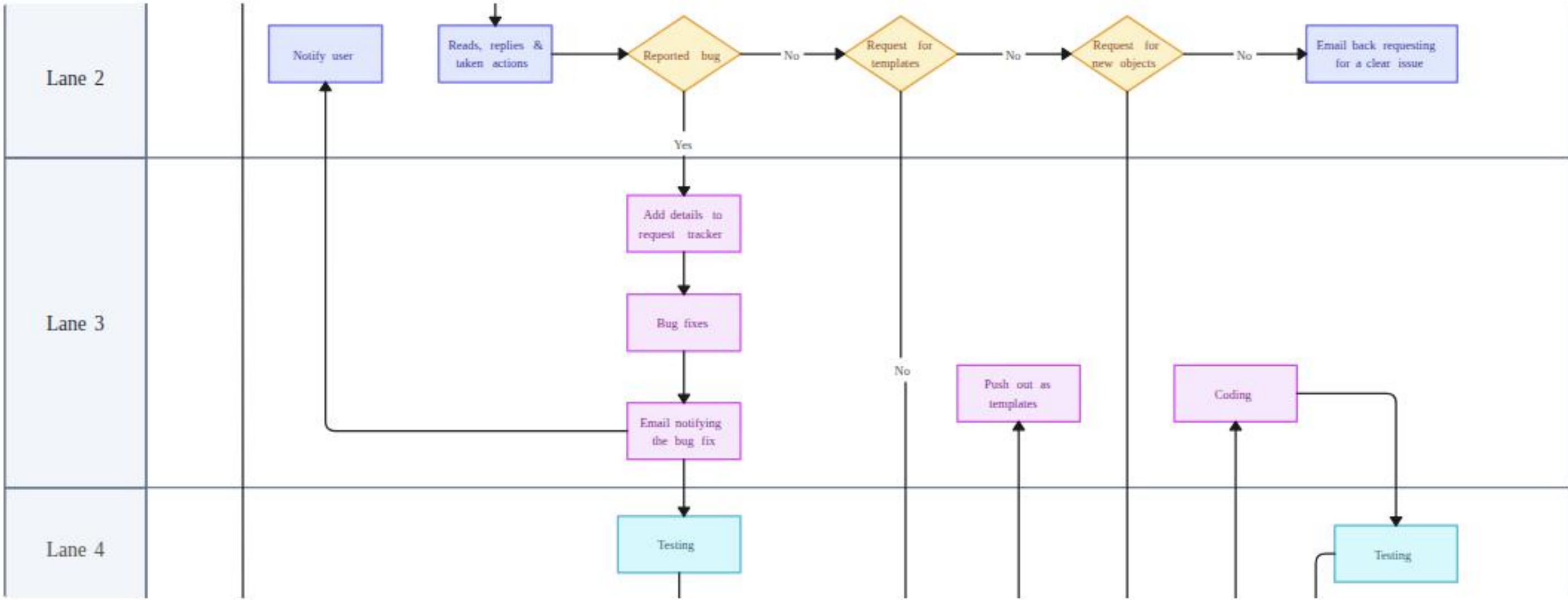
Functions that treat other functions as data (e.g., Map, Filter, Reduce).

Predictability Metrics

100%
DETERMINISTIC OUTPUT

Why it matters

Functional programming minimizes bugs by eliminating hidden states. It makes unit testing straightforward because every operation is isolated and verifiable.

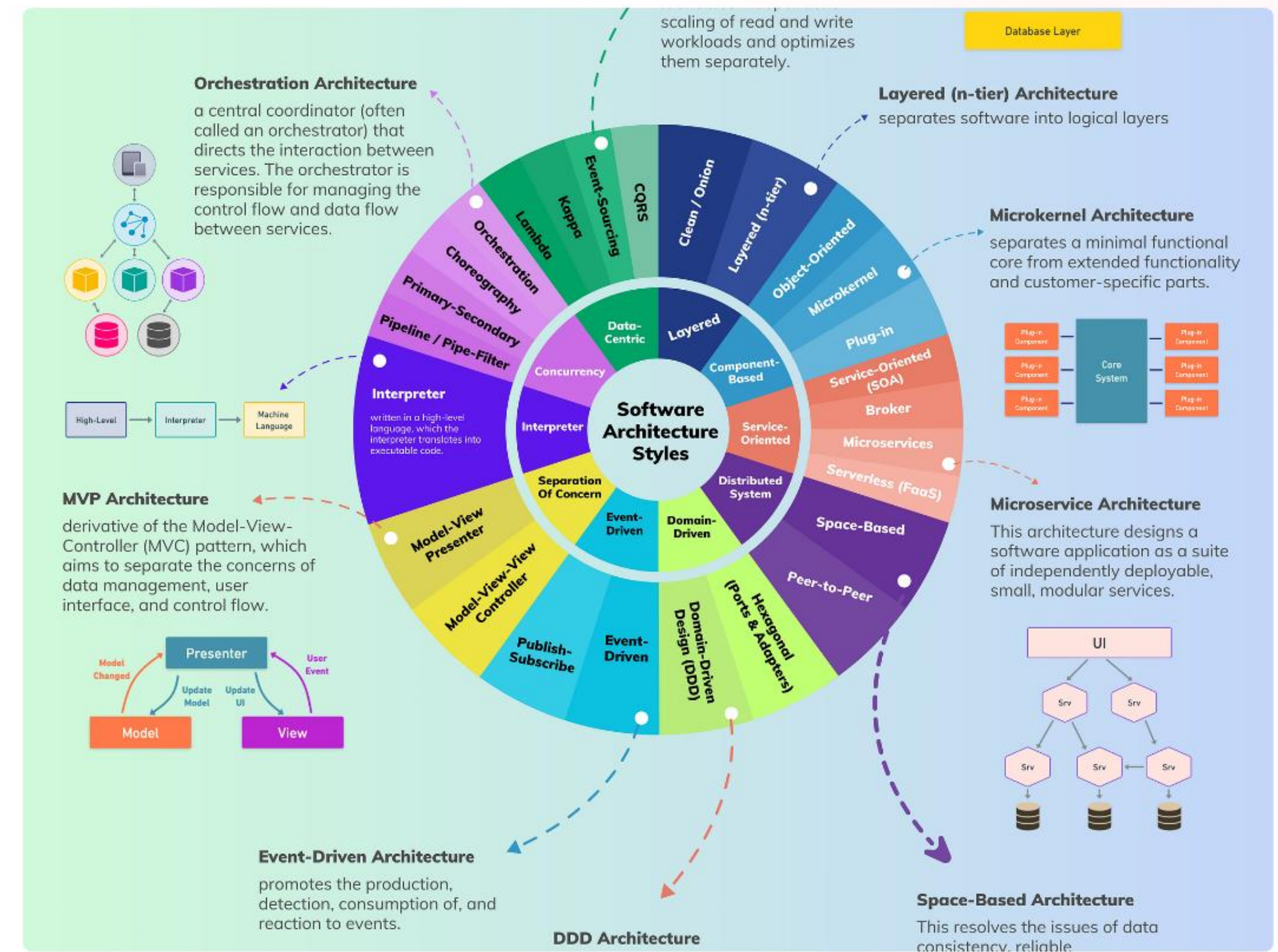


03. OBJECT-ORIENTED PROGRAMMING

Modeling complex systems through encapsulated state and behavior.

Classes and Objects

- > Blueprints (Classes): Define the structure and properties of potential objects.
- > Instances (Objects): Specific realizations of a class with their own unique data.
- > Methods: Encapsulated behaviors that operate on the object's internal data.



The Four Pillars of OOP



Encapsulation

Bundling data and methods together, hiding the internal implementation.



Inheritance

Deriving new classes from existing ones to promote code reuse.



Abstraction

Hiding complexity by exposing only the essential features of an object.



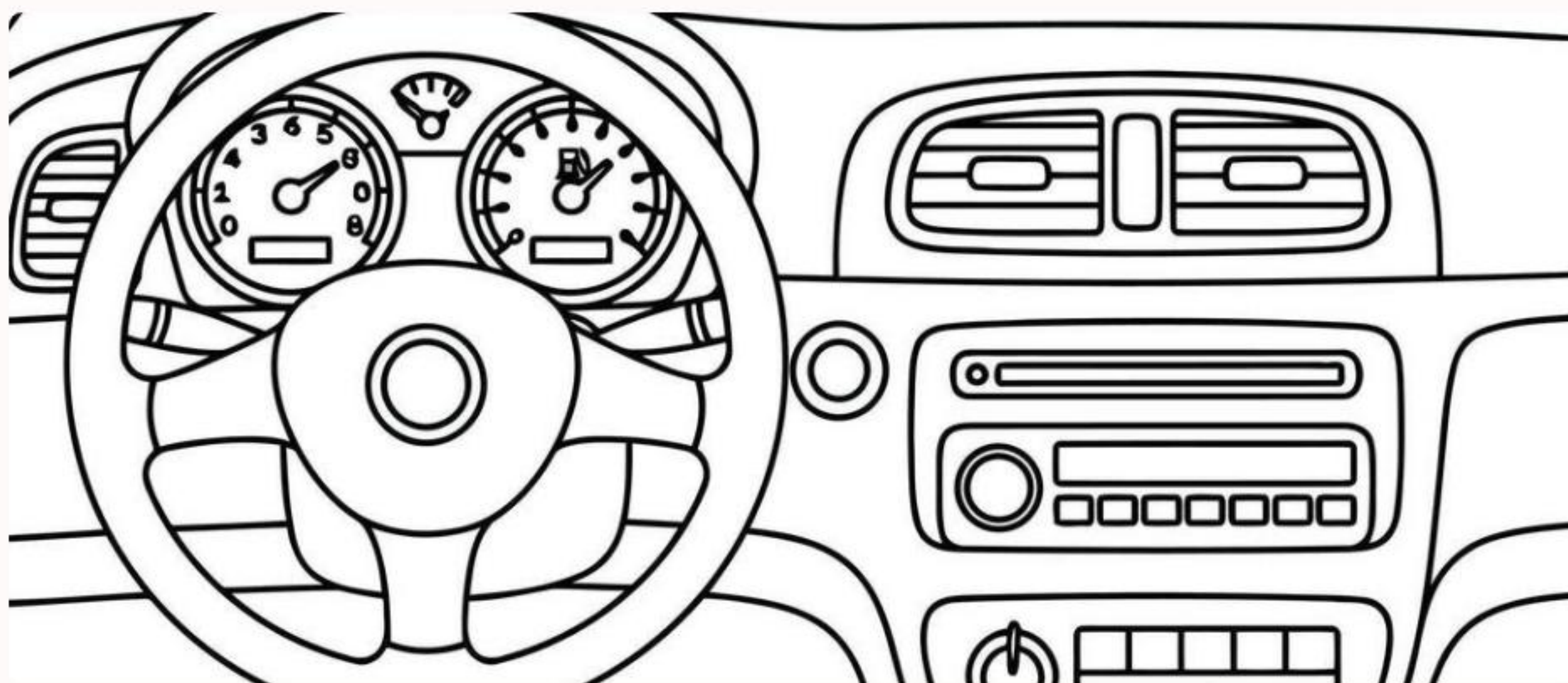
Polymorphism

Allowing different classes to be treated as instances of a same parent class.

Focus: Abstraction

Defining Complexity

Abstraction is about creating simple interfaces for complex operations. Think of a car dashboard: you see a speedometer and fuel gauge, but not the mechanical details of the engine.



Architectural Benefit

- Reduces cognitive load for developers using your code.
- Allows for implementation changes without affecting end-users.
- Ensures focus remains on "what" an object does rather than "how."

QUESTIONS?

Thank you for your attention.

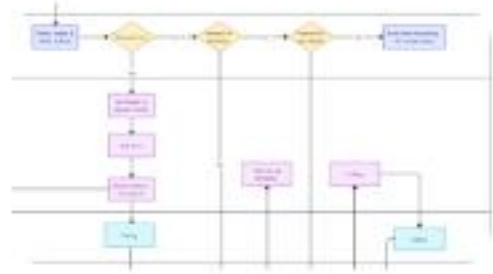
www.architecture-docs.com | dev-support@paradigm.com

Image Sources



https://assets.qlik.com/image/upload/w_1408/q_auto/qlik/glossary/data-analytics/seo-hero-data-mining_mop8cd.jpg

Source: www.qlik.com



<https://creately.com/static/assets/guides/types-of-flowcharts/support-process-swimlane-flowchart.svg>

Source: creately.com



https://miro.medium.com/v2/resize:fit:2000/1*46JX-o6xxMaMmOmTLwyLYQ.png

Source: medium.com



https://static.vecteezy.com/system/resources/previews/070/999/742/non_2x/serene-line-art-drawing-of-a-cars-interior-dashboard-steering-wheel-and-controls-in-a-simple-clean-style-vector.jpg

Source: www.vecteezy.com